



Integrazione Protocollo CM

Guida tecnica per sviluppatori — REST API + WebSocket

Informazioni sul documento

Questa guida è destinata agli sviluppatori che devono integrare il cassetto **CM** direttamente tramite il protocollo nativo, senza utilizzare il middleware CM Easy. Il documento descrive, passo per passo, l'intera sequenza di comunicazione: autenticazione JWT, connessione WebSocket, invio comandi e ricezione eventi.

Tutti gli importi sono espressi come **interi in centesimi di Euro** (es. 5,00 € = 500). Il campo *requestId* è opzionale: se omissso, *cm_driver* genera automaticamente un identificatore interno. Se fornito, può essere qualsiasi stringa univoca (non è richiesto il formato UUID); *cm_driver* lo riporta inalterato in tutti gli eventi di risposta al comando, permettendo al POS di correlare richieste ed eventi.

- **Canale principale:** WebSocket *ws://<host>:<porta>/ws* — comandi operativi e ricezione eventi in tempo reale.
- **Canale secondario:** REST API *http://<host>:<porta>/* — autenticazione, stato, configurazioni e report.
- **Autenticazione:** JWT — il token ottenuto via REST va incluso in ogni richiesta WebSocket e REST.

1. Autenticazione — Ottenimento Token JWT

Prima di qualsiasi operazione, il client deve autenticarsi tramite un endpoint REST per ottenere il token JWT. Questo token deve essere incluso nell'header *Authorization* di tutte le chiamate successive (REST e WebSocket).

Diagramma di flusso — Autenticazione



1. Codifica credenziali in Base64 *base64(username:password)*

--- *POST /auth/login* ----->

Header: *Authorization: Basic <base64>*

<-- *200 OK { "token": "eyJ..." } -----*

2. Salva il token JWT

Token valido per tutta la sessione

Richiesta HTTP

```
POST /auth/login
Authorization: Basic dXNlcm5hbWU6cGFzc3dvcmQ=
# base64("username:password")
```

Risposta HTTP 200 OK

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxfQ.abc123"
}
```

■ Se il token è già associato all'utente e non è scaduto, il server lo restituisce direttamente senza generarne uno nuovo.

■ Endpoint logout: **GET /auth/logout** (header: *Authorization: Bearer <token>*).

2. WebSocket — Connessione e Handshake

Dopo aver ottenuto il token JWT, il client deve aprire una connessione WebSocket all'endpoint `/ws`. Il token va incluso nell'header HTTP durante l'upgrade della connessione. Una volta stabilita la connessione, il client deve inviare immediatamente un messaggio di **handshake** per identificarsi.

Diagramma di flusso — Connessione WebSocket



Messaggio di Handshake (primo messaggio da inviare)

```
// Tipo messaggio: deve essere il PRIMO messaggio inviato dopo la connessione
{
  "type": "handshake",
  "clientId": "POS-001", // identificativo univoco del client
  "clientName": "Cassa Principale", // nome leggibile
  "clientUserCode": "OP-42", // codice operatore (opzionale)
  "clientUserName": "Mario Rossi", // nome operatore (opzionale)
  "websocketClientType": "client", // "client" per POS/casse
  "force": false // true = forza disconnessione client precedente
}
```

Tipo Client	websocketClientType	Descrizione
client	client	POS / cassa — può inviare comandi e ricevere eventi
display	display	Display informativo — solo ricezione (non può inviare comandi)
demon	demon	Monitor leggero — ricezione eventi filtrati (crediti, errori, start/end)

3. Struttura dei Comandi WebSocket

Tutti i comandi sono messaggi JSON inviati dal client (POS) al server tramite il canale WebSocket. Ogni comando deve avere un **requestId** UUID v4 univoco che permette di correlare le risposte agli eventi successivi.

Struttura del messaggio di comando

```
{
  "type": "command", // costante
  "requestId": "550e8400-e29b-41d4-a716-446655440000", // UUID v4
  "code": 22, // codice numerico del comando
  "name": "sale", // nome del comando
  "parameters": { "amount": 500 } // parametri specifici del comando
}
```

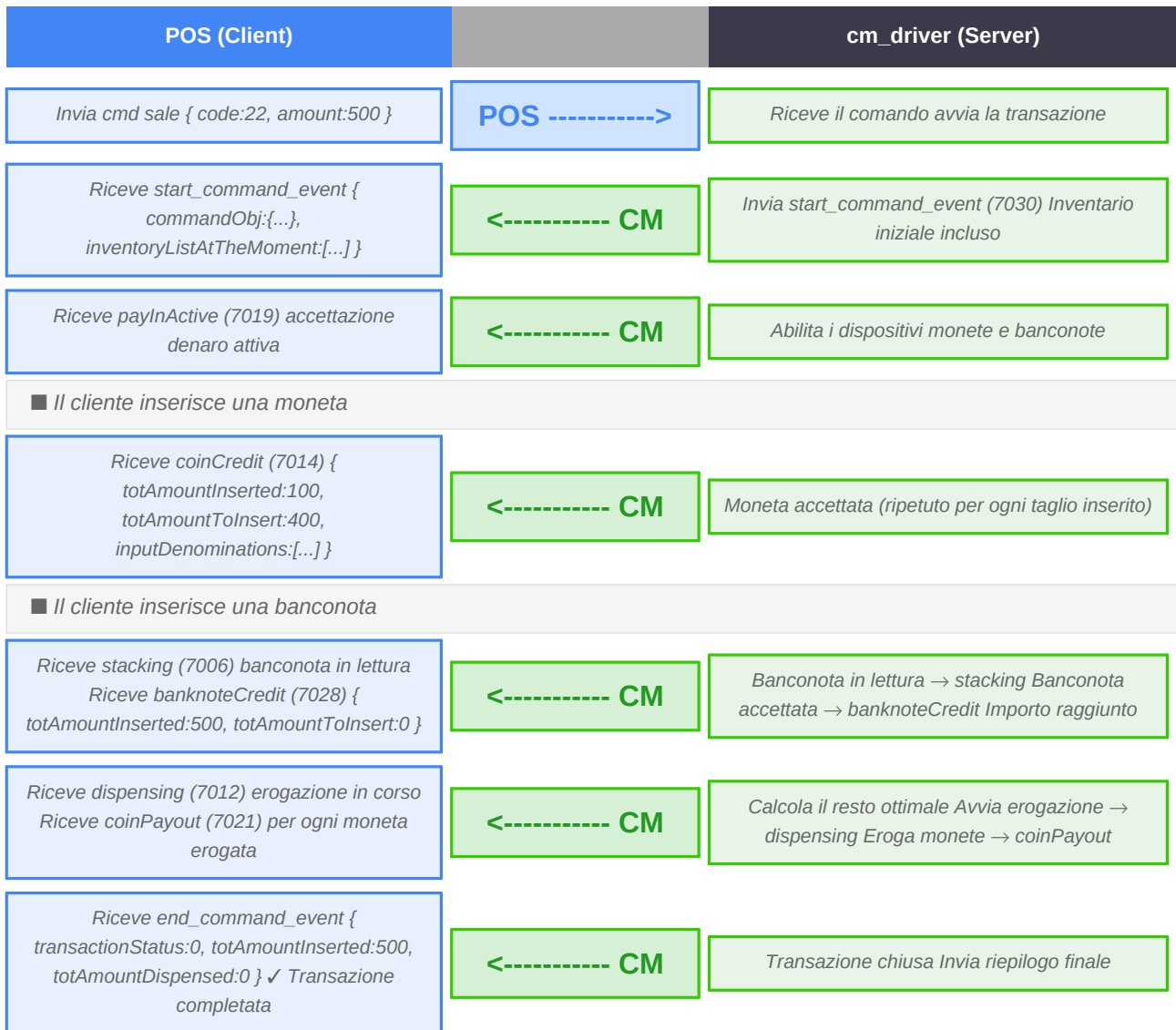
Elenco completo dei comandi disponibili

Codice	Nome	Parametri	Descrizione
20	payoutAmount	amount: int	Eroga un importo specificato (centesimi)
22	sale	amount: int	Transazione di vendita: accetta denaro, calcola e restituisce il resto
24	cancelSaleTransaction	—	Annulla la transazione corrente e restituisce il denaro inserito
30	startPayIn	—	Avvia modalità accettazione libera (senza importo target)
31	stopPayIn	—	Ferma la modalità accettazione libera
40	floatAmount	amount: int (opz.)	Trasferisce denaro nella cashbox per raggiungere il float
50	skimToMinimums	—	Preleva nella cashbox finché l'inventario è ai livelli minimi
100	empty	—	Svuota completamente il cassetto (monete + banconote)
101	emptyCashbox	—	Svuota la cassetta banconote (stacker)
102	replenish	denominations: list	Carica il cassetto con le denominazioni specificate
200	status	—	Richiede lo stato corrente del cassetto (risposta sincrona via evento)
255	reset	—	Reset di tutti i dispositivi hardware

4. Flusso Principale — Transazione di Vendita (sale)

Il comando **sale** è il cuore dell'integrazione POS. Il client invia l'importo da incassare; cm_driver abilita i dispositivi, gestisce l'inserimento di monete e banconote, calcola il resto ottimale e lo eroga automaticamente.

Diagramma di flusso — Transazione di Vendita



Esempio: invio comando sale

```
{
  "type": "command",
  "requestId": "a1b2c3d4-1234-5678-abcd-ef0123456789",
  "code": 22,
  "name": "sale",
  "parameters": { "amount": 1250 } // 12,50 €
}
```

5. Struttura degli Eventi Ricevuti

Il server invia eventi JSON al client WebSocket per notificare ogni cambiamento di stato del cassetto. Il campo **requestId** (o **requestID**) permette di associare l'evento al comando che lo ha generato.

5.1 — start_command_event (inizio comando)

```
{
  "type": "start_command_event",
  "requestId": "a1b2c3d4-...", // stesso requestId del comando
  "commandObj": { "code": 22, "name": "sale", ... },
  "inventoryListAtTheMoment": [...] // inventario al momento dell'avvio
}
```

5.2 — Evento di credito (es. coinCredit / banknoteCredit)

```
{
  "type": "event",
  "eventId": "uuid-evento",
  "code": 7014, // 7014=coinCredit, 7028=banknoteCredit
  "name": "coinCredit",
  "requestId": "a1b2c3d4-...",
  "commandType": "sale",
  "deviceType": "COIN", // "COIN" o "NOTE"
  "totAmountRequested": 1250, // importo totale richiesto (centesimi)
  "totAmountInsertedAtTheMoment": 100, // inserito finora
  "totAmountToInsertAtTheMoment": 1150, // ancora da inserire
  "inputDenominations": [{ "value":100, "quantity":1, "type":"COIN" }]
}
```

5.3 — end_command_event (fine comando)

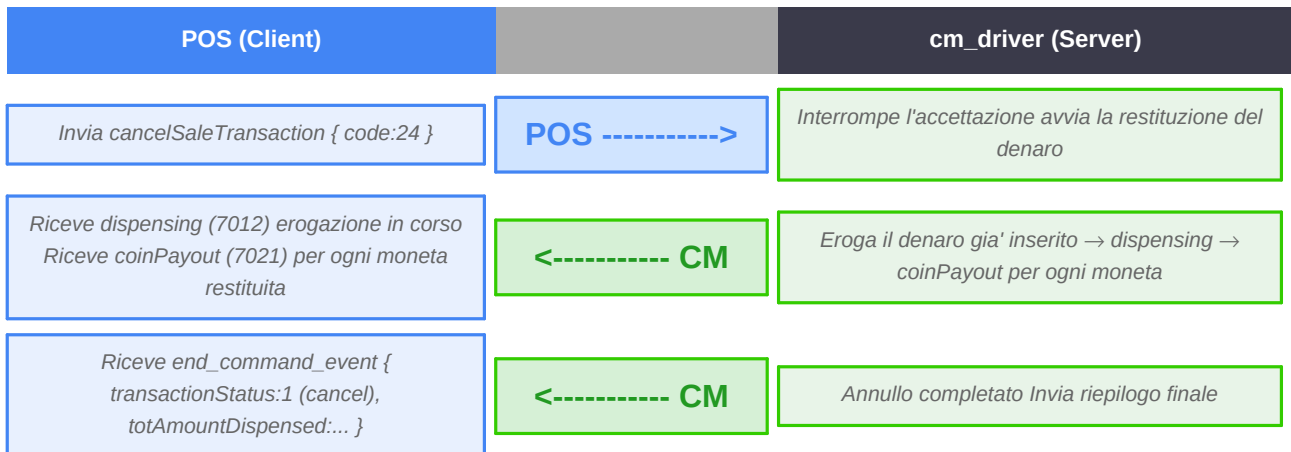
```
{
  "type": "end_command_event",
  "requestId": "a1b2c3d4-...",
  "commandObj": { "code": 22, "name": "sale" },
  "transactionStatus": 0, // 0=success, 1=cancel, 10=changeShortage...
  "totAmountRequested": 1250,
  "totAmountInsertedAtTheMoment": 2000, // inserito dal cliente
  "totAmountDispensedAtTheMoment": 750, // resto erogato
  "transactionDuration": 45, // durata in secondi
  "inputDenominations": [...], // tagli inseriti
}
```

```
"outputDenominations": [...], // tagli erogati come resto
"inventoryDenominations": [...] // inventario aggiornato a fine transazione
}
```

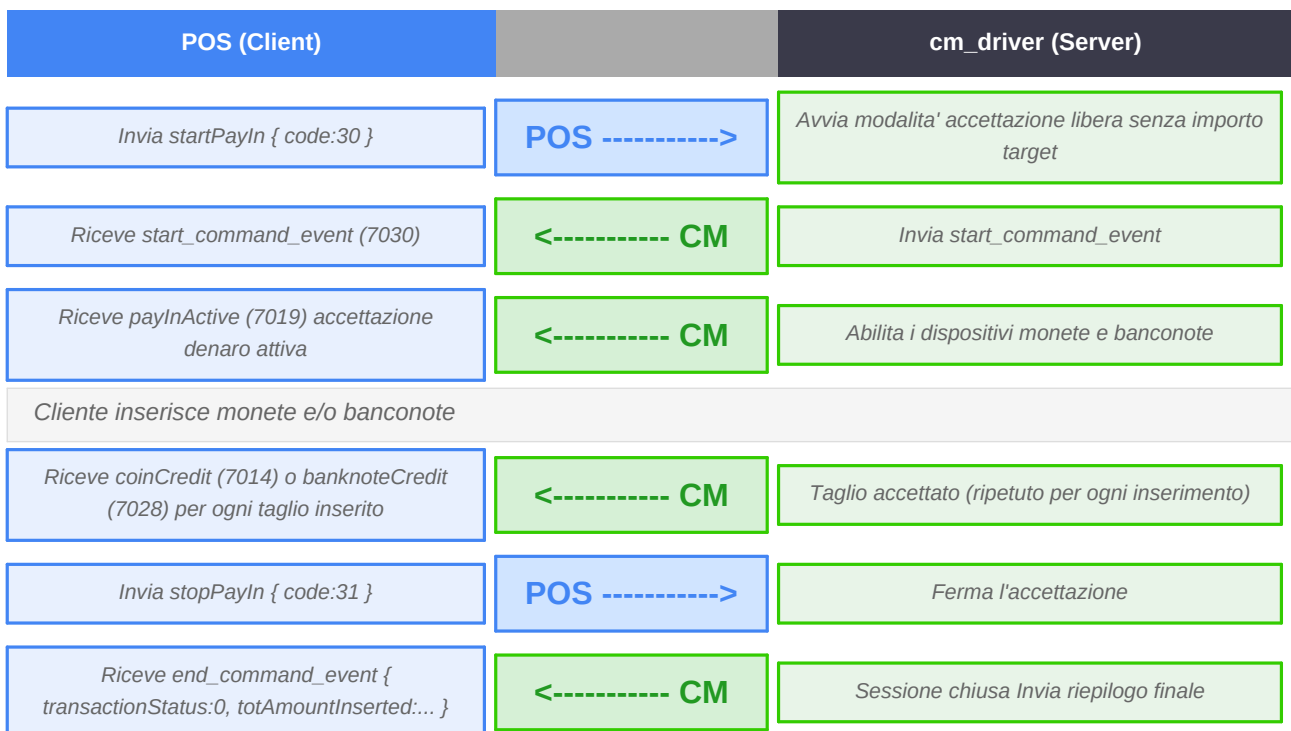
transactionStatus	Codice	Descrizione
success	0	Transazione completata con successo
cancel	1	Transazione annullata (denaro restituito)
reset	2	Terminata a seguito di reset macchina
cancelChangeShortage	9	Annullata — non è stato possibile restituire tutto il denaro
changeShortage	10	Completata ma resto insufficiente (erogato parzialmente)
dispensedChangeInconsistency	12	Errore nell'erogazione del resto
deviceError	100	Errore hardware (inceppamento o dispositivo non risponde)
genericError	999	Errore generico

6. Altri Flussi Operativi

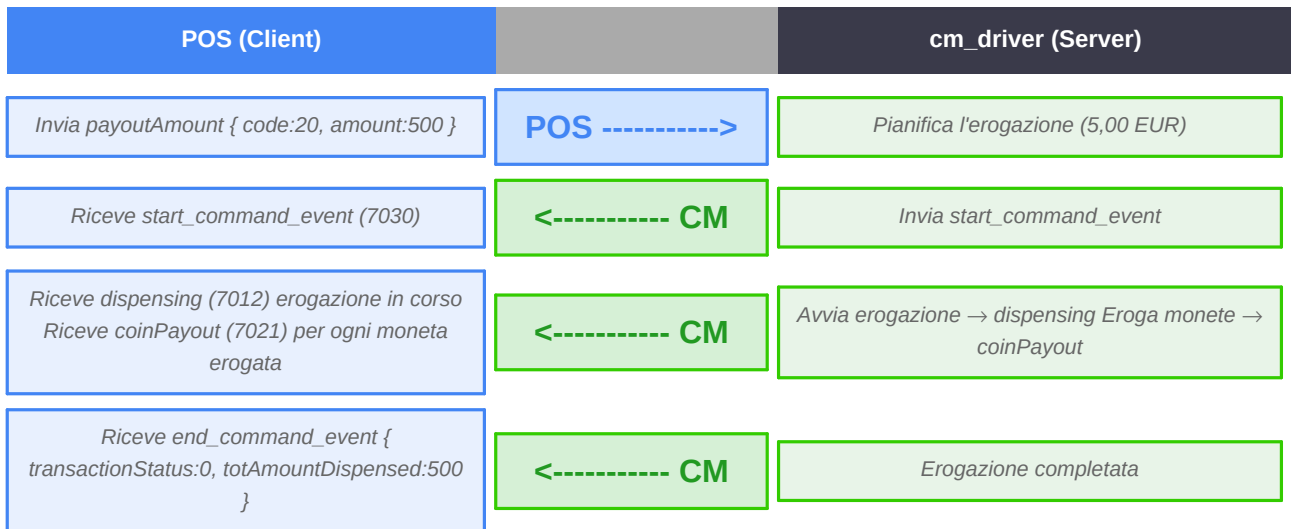
6.1 — Annulla Transazione (cancelSaleTransaction)



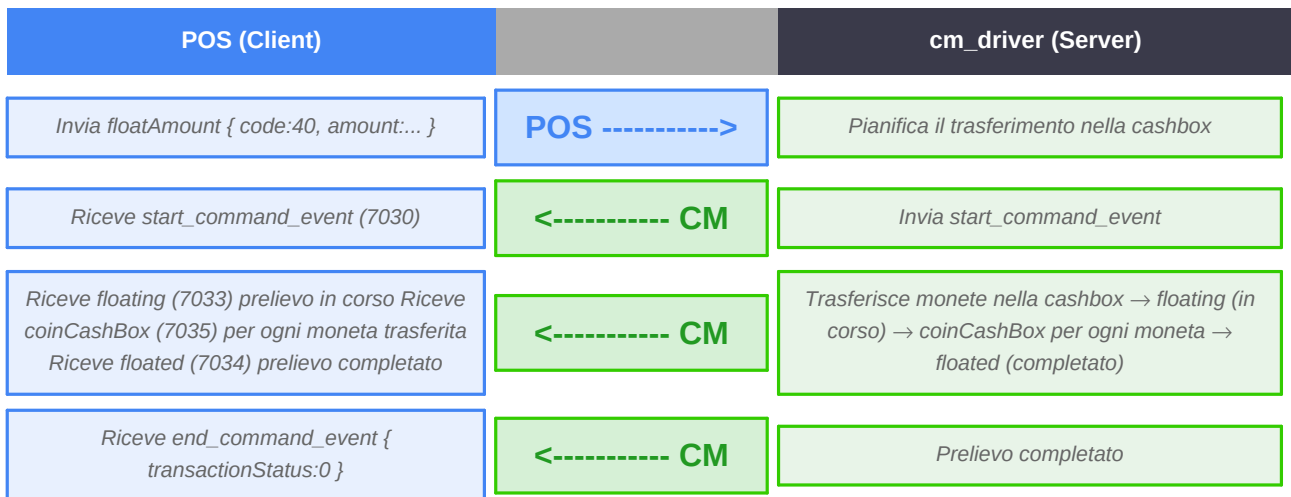
6.2 — Accettazione Libera (startPayIn / stopPayIn)



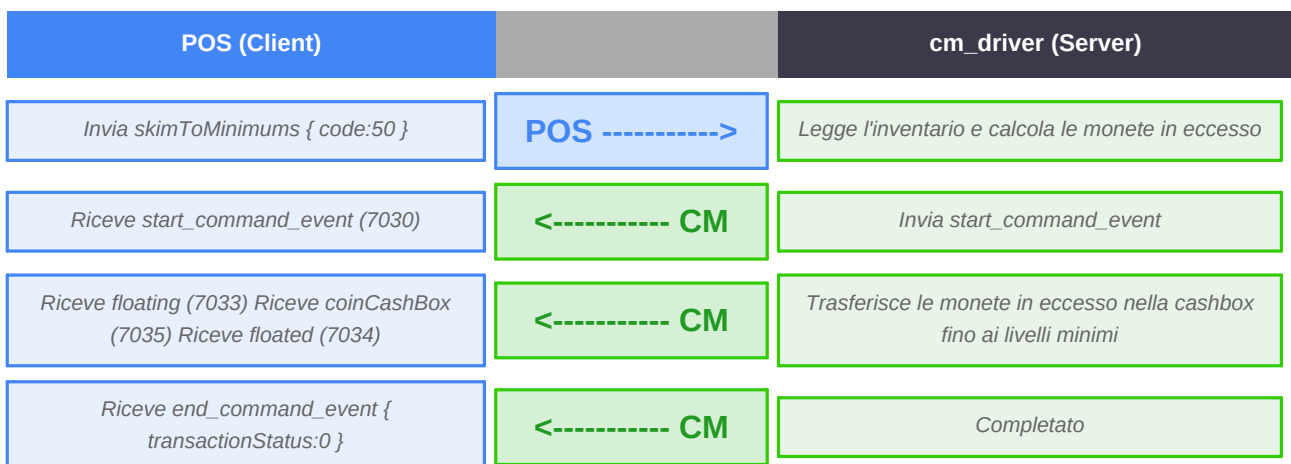
6.3 — Erogazione Diretta (payoutAmount)



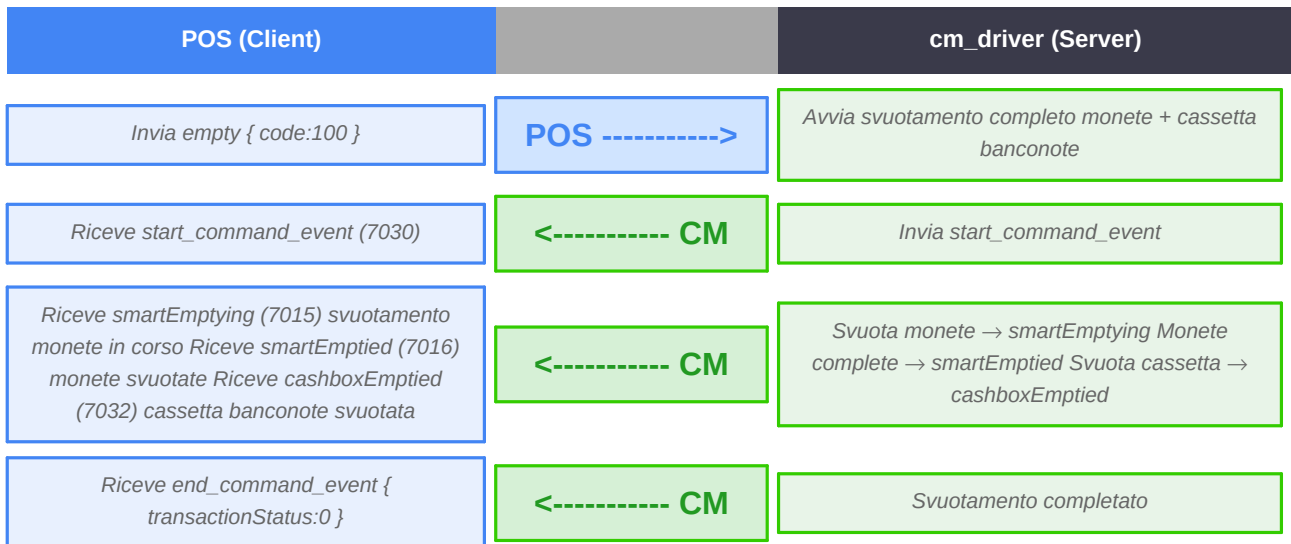
6.4 — Prelievo Float (floatAmount)



6.5 — Preleva ai Minimi (skimToMinimums)



6.6 — Svuotamento Completo (empty)



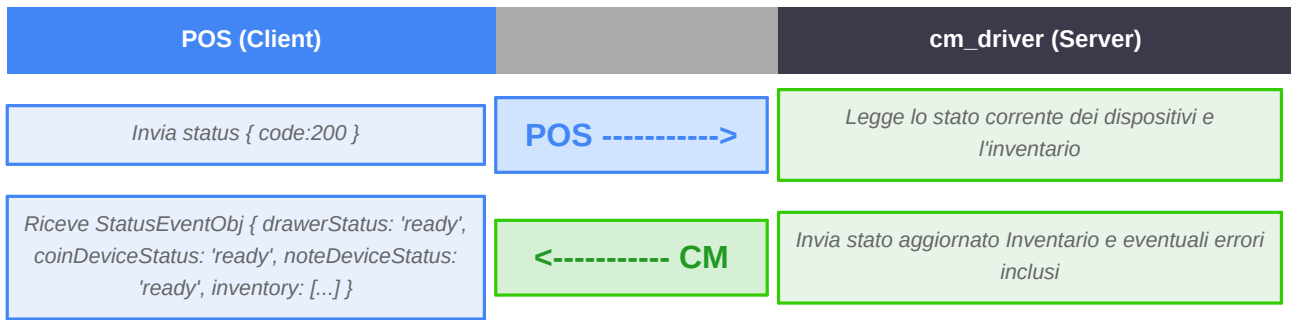
6.7 — Svuotamento Cassetta Banconote (emptyCashbox)



6.8 — Caricamento Cassetto (replenish)



6.9 — Stato Cassetto (status)



6.10 — Reset Dispositivi (reset)



7. Riferimento Completo — Eventi WebSocket

Tabella di tutti gli eventi che il server può inviare al client. La colonna *Transaz.* indica se l'evento ha rilevanza transazionale (crediti, erogazioni, stato inventario).

Codice	Nome evento	Transaz.	Descrizione
0	idle	Si	Cassetto in attesa, nessuna operazione in corso
7001	slaveReset	No	Il cassetto ha completato il reset del dispositivo
7002	read	No	Il lettore banconote sta leggendo una banconota
7004	rejecting	No	Il lettore sta rifiutando una banconota
7005	rejected	Si	Banconota rifiutata (valore in rejected + denominazione)
7006	stacking	No	La banconota sta entrando fisicamente nel dispositivo
7007	stacked	No	La banconota è stata inserita nel dispositivo
7008	cashboxReplaced	Si	Cassetta banconote reinserita
7009	barcodeTicketValidated	No	Biglietto con barcode letto e validato
7010	barcodeTicketAck	No	Biglietto barcode inserito nello stacker
7011	initialising	No	Il cassetto si sta inizializzando
7012	dispensing	No	Il cassetto sta erogando denaro
7013	dispensed	No	Erogazione completata
7014	coinCredit	Si	Moneta accettata (aggiorna totAmountInserted)
7015	smartEmptying	No	Svuotamento monete in corso
7016	smartEmptied	No	Svuotamento monete completato
7017	noteTransferredToStacker	Si	Banconota trasferita nello stacker (cashbox)
7018	noteHeldInBezel	No	Banconota espulsa in attesa di ritiro
7019	payInActive	No	Modalità accettazione denaro attiva
7020	maintenanceRequired	No	Il cassetto richiede manutenzione
7021	coinPayout	Si	Monete erogate (outputDenominations aggiornato)
7022	initialized	No	Cassetto inizializzato correttamente
7023	disabled	No	Dispositivo disabilitato
7026	notInitialized	No	Inizializzazione fallita
7027	storedInCashbox	Si	Banconota inviata nella cashbox (non nel riciclatore)
7028	banknoteCredit	Si	Banconota accettata (aggiorna totAmountInserted)

Codice	Nome evento	Transaz.	Descrizione
7029	storedInPayout	No	Banconota inviata nel payout (riciclatore)
7030	startCommand	Si	Comando avviato — contiene inventario iniziale
7031	endCommand	Si	Comando terminato — contiene risultato transazione
7032	cashboxEmptied	Si	Cassetta banconote svuotata
7033	floating	No	Prelievo float in corso
7034	floated	Si	Prelievo float completato
7035	coinCashBox	Si	Monete inviate nella cashbox
7100	giveChange	No	Calcolo resto in corso
7400	noteTransferredToRcTray	Si	Banconota trasferita nel vassoio di riciclo (NV4000)
7401	replenishmentCassetteRemoved	Si	Cassetta di ricarica rimossa (NV4000)
7402	replenishmentCassetteReplaced	Si	Cassetta di ricarica reinserita (NV4000)
7403	replenishing	No	Ricarica in corso (NV4000)
7404	replenished	Si	Ricarica completata (NV4000)

8. Gestione Errori WebSocket

Quando si verifica un errore, il server invia un messaggio JSON con **type: "error"**. L'errore contiene il codice e il nome dell'errore, il requestId del comando che lo ha generato (se applicabile) e un eventuale sub-errore.

```
{
  "type": "error",
  "code": -1005, // codice errore
  "name": "commandCannotBeProcessed",
  "requestId": "a1b2c3d4-...", // requestId del comando che ha causato l'errore
  "subError": "drawerBusy" // sub-errore (opzionale)
}
```

Codice	Nome errore	Causa
-1001	commandNotKnown	Il comando inviato non è riconosciuto
-1002	commandNotSupported	Comando non supportato dal modello di cassetto configurato
-1003	wrongNumberOfParameters	Numero di parametri errato nel comando
-1004	parametersOutOfRange	Uno o più parametri sono fuori range
-1005	commandCannotBeProcessed	Comando non eseguibile in questo momento (es. cassetto in errore)
-1006	softwareError	Errore software interno
-2001	busy	Un altro client ha già un comando in esecuzione
-2002	drawerDisabled	Il cassetto è disabilitato — abilitarlo prima con enable
-2003	clientNotYetPresented	Il client non ha ancora inviato l'handshake
-2004	wrongPayloadFormat	Il JSON del comando non è valido o manca di campi obbligatori
-3001	deviceNotInitialized	Il dispositivo hardware non si è inizializzato correttamente
-3002	mismatchDenominations	Disallineamento tra tagli attesi e tagli rilevati

■■ **Gestione del busy:** se si riceve l'errore *busy* (-2001), attendere l'evento **end_command_event** prima di inviare un nuovo comando. Non è necessario disconnettersi e riconnettersi.

■■ **Ping/Pong:** il server invia automaticamente un ping ogni 30 secondi. Il client WebSocket standard risponde con pong automaticamente. In assenza di risposta il server chiude la connessione.

9. REST API — Riferimento Endpoint

Tutti gli endpoint REST (tranne **/auth/login**) richiedono il token JWT nell'header: **Authorization: Bearer <token>**. Le risposte sono in formato JSON.

9.1 — Autenticazione

Metodo	Endpoint	Descrizione
POST	/auth/login	Login — credenziali via header Basic Auth; risposta: { "token": "..."
PUT	/auth/login	Cambio password — body JSON con username e password in base64
GET	/auth/logout	Logout (no-op server-side; il token scade per expiry naturale)

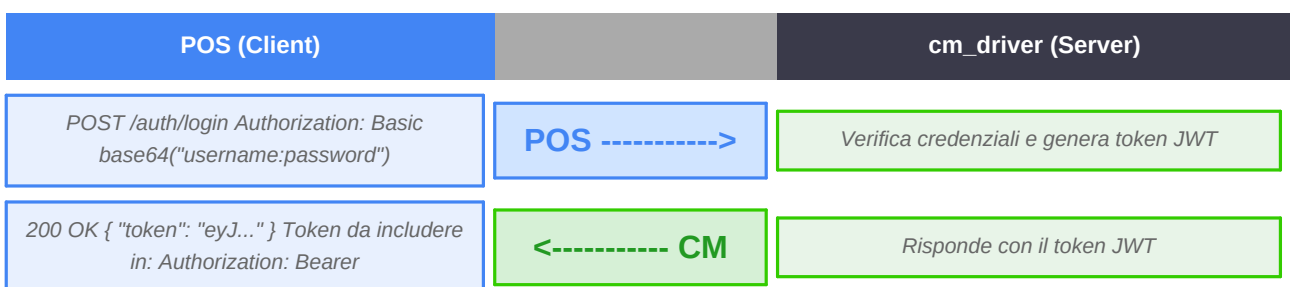
Payload **POST /auth/login** — nessun body; credenziali nell'header HTTP:

```
Authorization: Basic base64("username:password")  
  
# Esempio:  
Authorization: Basic YWRtaW46cGFzc3dvcmQ=
```

Payload **PUT /auth/login** — le password devono essere codificate in base64:

```
{  
  "username": "admin",  
  "oldPassword": "base64(vecchiaPassword)",  
  "newPassword": "base64(nuovaPassword)"  
}
```

Flusso — POST /auth/login

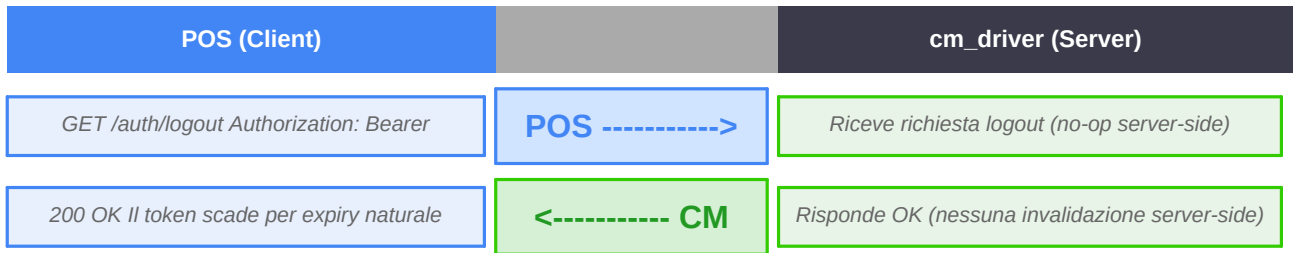


Flusso — PUT /auth/login (cambio password)





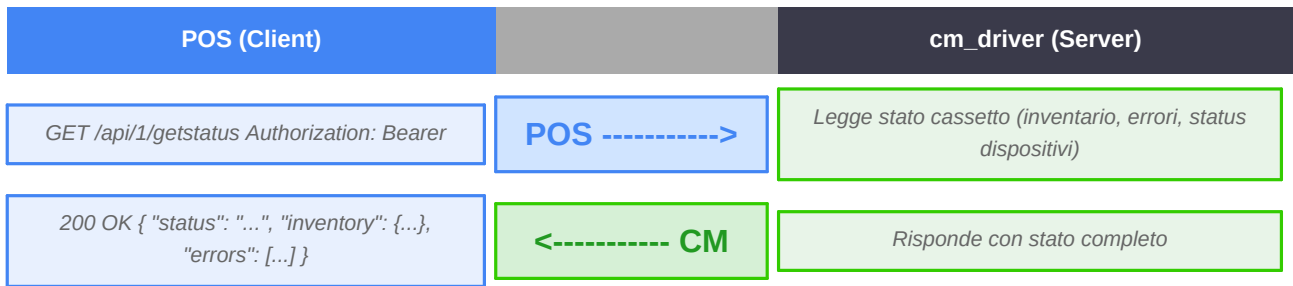
Flusso — GET /auth/logout



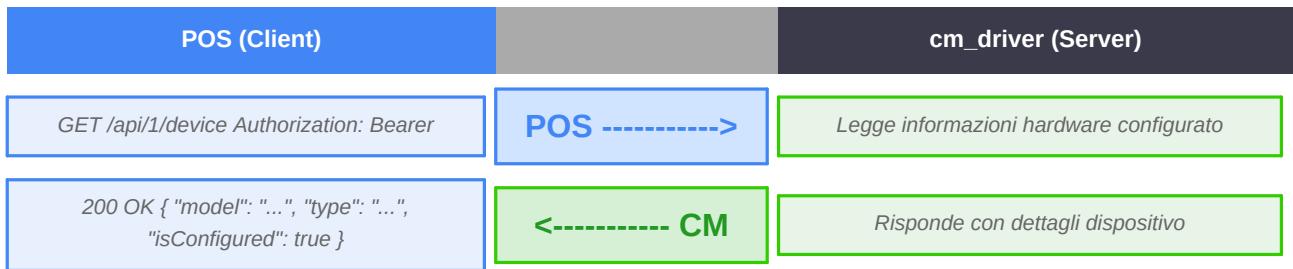
9.2 — Stato e Controllo

Metodo	Endpoint	Descrizione
GET	/api/1/getstatus	Stato attuale del cassetto (inventario, errori, status dispositivi)
GET	/api/1/device	Informazioni sul dispositivo hardware configurato
GET	/deviceinfo	Informazioni hardware (endpoint pubblico, no token)
POST	/api/1/reboot	Riavvia il sistema — 3 modalita' mutuamente esclusive (vedi payload)

Flusso — GET /api/1/getstatus



Flusso — GET /api/1/device

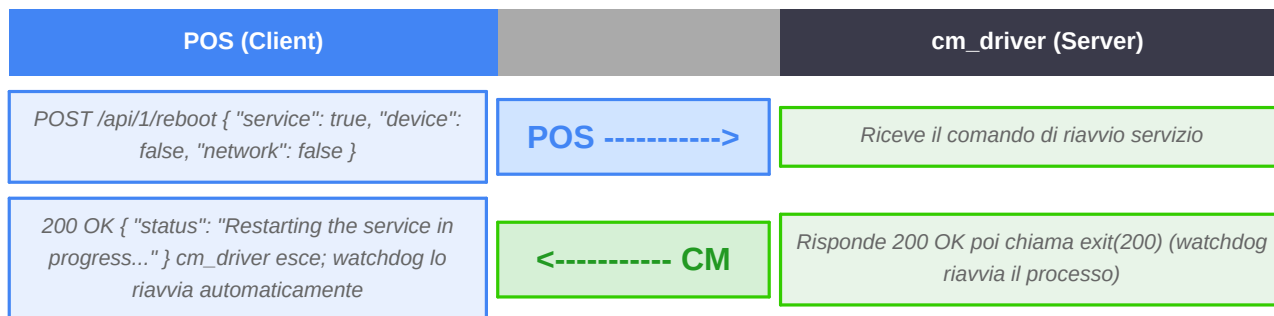


Flusso — GET /deviceinfo (no token)

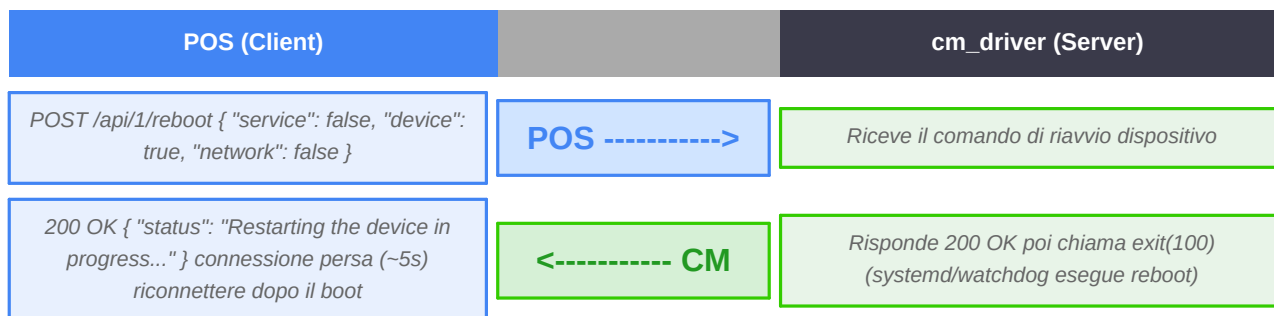




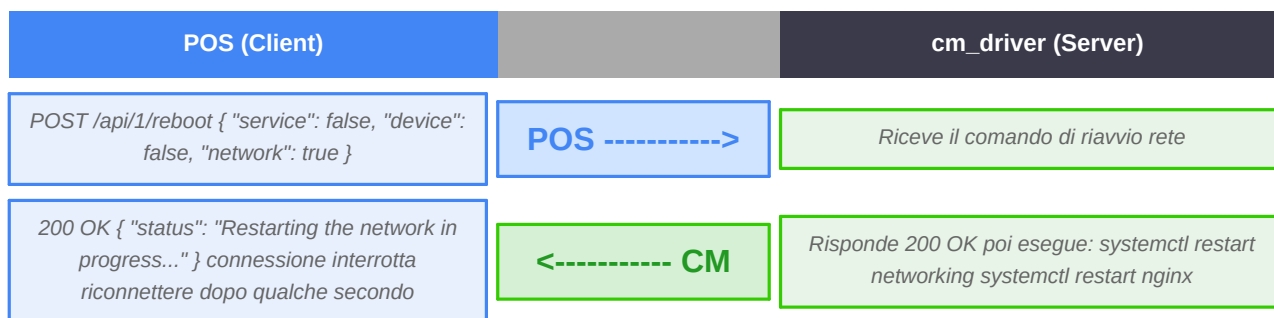
Flusso — Riavvio processo cm_driver



Flusso — Riavvio fisico PC / Raspberry Pi



Flusso — Riavvio servizi di rete

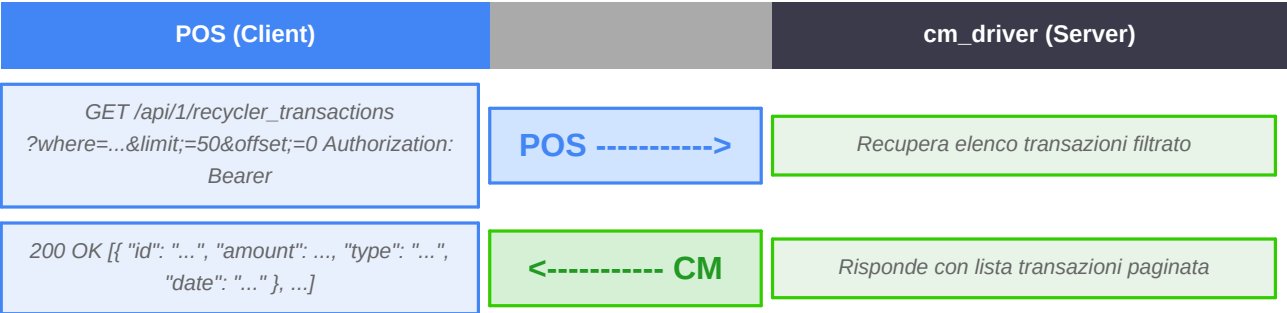


9.3 — Report e Dati

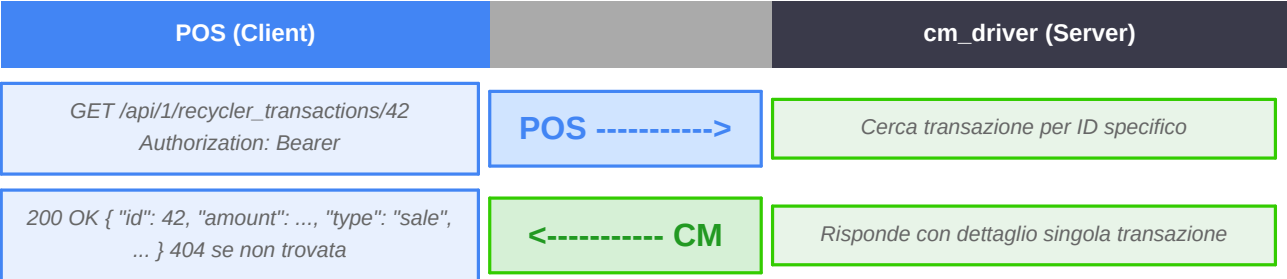
Metodo	Endpoint	Descrizione
GET	/api/1/recycler_transactions	Elenco transazioni (query param opzionali: where, limit, offset)
GET	/api/1/recycler_transactions/	Singola transazione per ID
GET	/api/1/recycler_transactions/date	Data dell'ultima transazione di tipo float

Metodo	Endpoint	Descrizione
GET	/api/1/recycler_inventory	Inventario attuale del cassetto
GET	/api/1/recycler_denominations	Configurazione denominazioni (valori accettati/riciclati)
GET	/api/1/recycler_denominations_journal	Journal delle modifiche alle denominazioni
GET	/api/1/report1	Report operativo (riepilogo movimenti)
GET	/api/1/report2	Report dettagliato (per periodo)

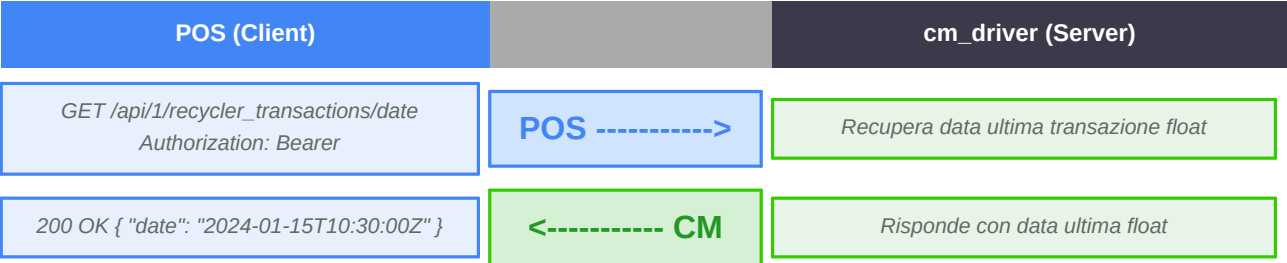
Flusso — GET /api/1/recycler_transactions



Flusso — GET /api/1/recycler_transactions/



Flusso — GET /api/1/recycler_transactions/date

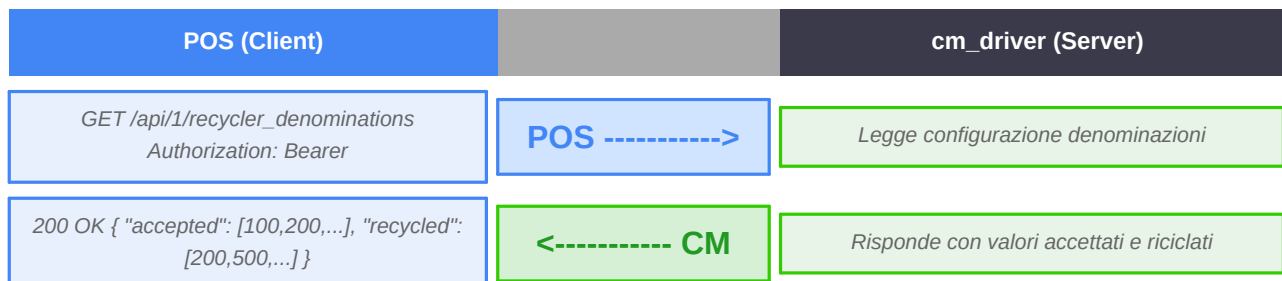


Flusso — GET /api/1/recycler_inventory

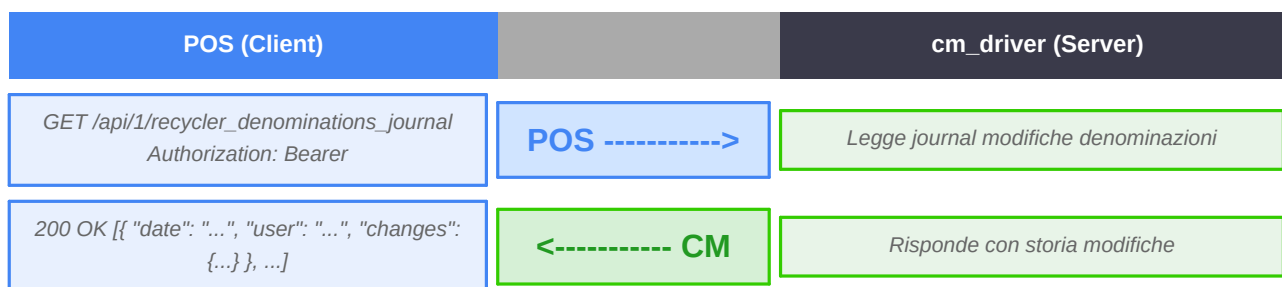




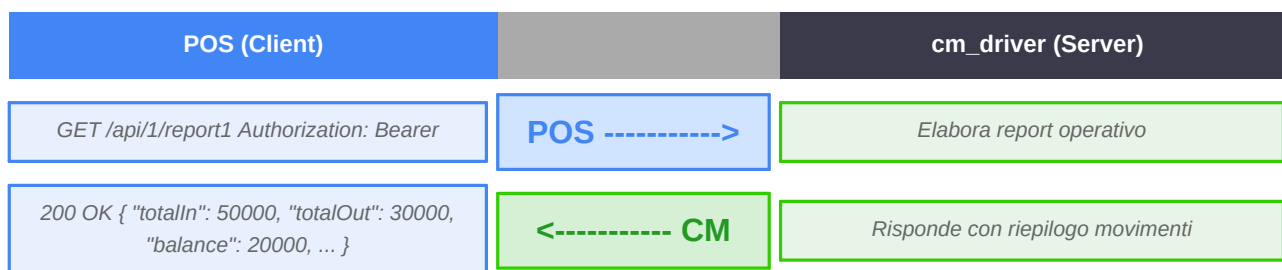
Flusso — GET /api/1/recycler_denominations



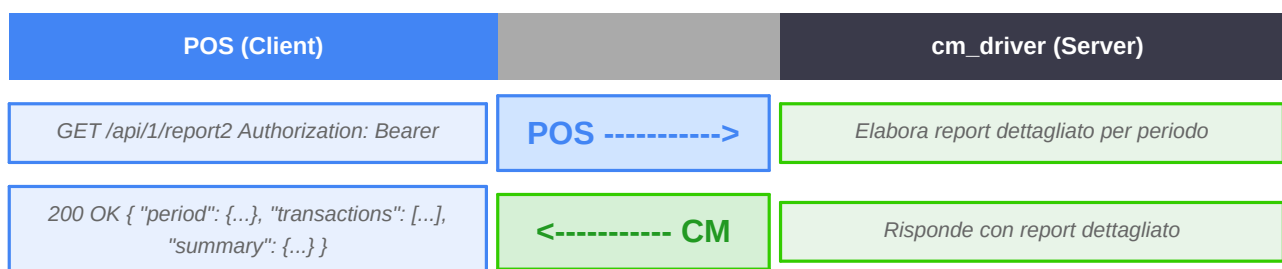
Flusso — GET /api/1/recycler_denominations_journal



Flusso — GET /api/1/report1



Flusso — GET /api/1/report2



9.4 — Gestione Utenti e Casse

Metodo	Endpoint	Descrizione
GET/POST/DELETE	/api/1/tokens	Token JWT (PUT -> 401; token non modificabile)

Flusso — Token JWT (/api/1/tokens)

POS (Client)		cm_driver (Server)
GET /api/1/tokens	POS ----->	Recupera token attivi
200 OK [{ "token": "...", "user": "...", "expires": "..." }]	<----- CM	Lista token attivi
POST /api/1/tokens { "username": "...", "password": "..." }	POS ----->	Genera nuovo token
200 OK { "token": "eyJ..." }	<----- CM	Token generato
DELETE /api/1/tokens/	POS ----->	Invalida token
200 OK { "status": "ok" }	<----- CM	Token invalidato
PUT /api/1/tokens (non supportato)	POS ----->	Operazione non permessa
401 Unauthorized { "error": "not allowed" }	<----- CM	Token non modificabile

10. Checklist di Integrazione

Sequenza minima per una corretta integrazione del cassetto CM nel sistema POS:

#	Step	Azione
1	Autenticazione	POST /auth/login → salva il JWT ricevuto
2	Connessione WS	Apri ws://:/ws con header Authorization: Bearer
3	Handshake	Invia { "type":"handshake", "clientId":"...", "websocketClientType":"client" }
4	Stato iniziale	Verifica lo stato con cmd status (code:200) o GET /api/1/getstatus
5	Transazione	Invia sale (code:22) con il parametro amount in centesimi
6	Ascolto eventi	Gestisci: start_command_event → coinCredit/banknoteCredit → end_command_event
7	Fine sessione	Invia GET /auth/logout e chiudi la connessione WebSocket

Risorse e Supporto

■ Documentazione Generale CM

<https://nemesix.github.io/nx-docs/documentazione-cm.html>

Panoramica completa del sistema e del protocollo.

■ API WebSocket — Specifica completa

<https://nemesix.github.io/nx-docs/cm-websocket/>

Dettaglio di tutti i comandi, eventi e gestione errori.

■ API REST — Riferimento endpoint

<https://nemesix.github.io/nx-docs/cm-restapi/>

Specifica OpenAPI degli endpoint REST.

■ Simulatore CM Connector — Download

https://nemesix.github.io/nx-docs/app/cm_connector_latest.zip

Simula il cassetto CM senza hardware fisico per lo sviluppo.

■ Manuale CM Connector (Simulatore)

<https://nemesix.github.io/nx-docs/documentazione-cm-connector.html>

Installazione, configurazione e scenari di test.

Supporto Tecnico

Per dubbi su logiche JSON o sul simulatore, contattare il team tecnico:

Email: cashmarket@nemesix.it

Nota: allegare sempre un estratto dei log (request/response) relativi al problema.